

Practical Techniques in Fundamental Software Development: Intelligent Software Engineering

Intelligent Software Engineering Experiment Report

Experiment 4: Improving Code Review for Code Generated by Large Models

Name	Marlen Melis	Department	School of Computing
Student ID	2026140428	Course Teacher	Liu Jiakun
Instructor	Liu Jiakun	Lab Location	Dormitory
Lab Time	Evening		
Lab Score		Discussion Score	
Report Score		Total Experiment Score	

1 Experiment Objectives

Upon completion of this experiment, I am able to:

1. Understand the primary reasons code review performance varies when applied to **AI-generated** code, as distinct from the human-written code Experiment 3 reviewed.
2. Master methods for constructing software-engineering context at different granularities — repository-level code context, linked-issue text, and historical review comments on similar past PRs — and understand their impact on review tasks.
3. Master prompt-engineering techniques that optimize code-review performance specifically for AI-generated code, including **self-reflection** (critique-then-finalize) and

multi-turn (progressive context disclosure) — reasoning mechanisms Experiment 3 did not use.

4. Design prompts suited to AI-generated code, carrying forward the three strategies (role-based, few-shot, Chain-of-Thought) that generalize from Experiment 3 while adding the two new ones above.
5. Analyze the impact of different prompt and context combinations on merge prediction and review-comment generation, with real, computed numbers.
6. Propose improvement strategies for reviewing AI-generated code, grounded in what the data actually shows rather than asserted in the abstract.

2 Experiment Content

1. Data Preparation

The lab guide’s own text (§4.7.1, §4.7.5, §4.4.1) repeatedly says to load data and compare results “from Experiment 5” — a guide-authoring artifact with no referent, since this course has only four experiments. Reading §4.4.1 in full resolves it unambiguously: *“Compared to Experiment 5, this experiment focuses on optimizing model inputs rather than modifying the model itself”* — that sentence describes exactly the Experiment-3-to-Experiment-4 relationship (Exp 3 used simple prompts on human code; Exp 4 optimizes context and prompts on AI code), so “Experiment 5” is resolved as **Experiment 3** throughout, the same category of self-referential-numbering typo Experiment 3’s own guide text had (§3.7.1, resolved in `reports/lab3.tex` §1). The population is the **AI-authored PR subset** of the Experiment-1 dataset (`is_ai_authored=True`): **380 PRs** (361 agent-authored by `copilot-swe-agent`, 19 human-opened-AI-assisted), split per-repo by time exactly as Experiment 2’s ML split was (`exp4_examples.load_ai_train_pool/load_ai_t` into **302 train** / **78 test** PRs, keeping few-shot demonstrations and historical-comment retrieval leakage-safe against the scored test set.

2. Context Construction (`src/llm/prompts/context_aug.py`, `src/llm/exp4_c`

Five context tiers per the guide’s §4.7.2 Step 2 — deliberately *not* Experiment 3’s four: **diff**; **diff + PR description**; **diff + repository context**; **diff + issue information**; **complete software-engineering context** (all of the above plus historical review comments). Two of these needed genuinely new engineering, since nothing in the mined dataset stores full file bodies or issue text:

- **Repository context** is a *no-fetch approximation*: co-changed files grouped by directory, plus the enclosing function/class git names from each diff hunk’s own header (present on 99% of AI-PR changed-file rows) — a real, free signal that needs no extra API call, explicitly labelled as an approximation rather than a real call graph.
- **Issue text** is resolved via a new `GitHubClient.fetch_issue` method (`src/mining/github_client.py`) narrowly scoped: only the single most-confident referenced issue number (a closing keyword or full URL, preferred over a bare `#N` mention) is fetched, and PR-shaped references are skipped.
- **Historical comments** on similar past PRs (`exp4_examples.select_historical_comments`) retrieve real human review comments from the **train**-split PRs in the same repository, ranked by changed-directory overlap with the query PR — leakage-safe by construction, since the query PR’s own review is never in the candidate pool.

Every slot a tier calls for but has no data for renders an explicit `"(not available)"` marker rather than silently vanishing.

3. Prompt Design (`src/llm/prompts/conversations.py`, `schema.py`)

Five strategies per the guide’s §4.7.3 Step 3 plus the §4.4.3 exploration it separately names: **role-based**, **few-shot**, **Chain-of-Thought** carried over *unchanged* from Experiment 3 (same builders, same output contracts, only the richer context and AI-authored population differ) — and two genuinely new ones, kept in a separate `Exp4Strategy` enum so Experiment 3’s completed grid/tests are untouched:

- **Self-reflection** *replaces* zero-shot in the guide’s concrete strategy list. Turn 1 drafts a review/decision over the full tier context; turn 2 asks the model to critique its own draft, explicitly tuned to AI-code failure modes (hallucinated APIs, missing edge cases, over-confident merge calls) before finalizing.
- **Multi-turn** discloses context progressively: turn 1 sees the diff alone; turn 2 reveals the tier’s broader context and asks the model to reconsider. Undefined on the bare-diff tier (nothing to reveal in a second turn), so that one cell is skipped by design, not by accident.

Both render to a `RenderedConversation` (a persona + an ordered tuple of turns) rather than a single-shot prompt, because their reasoning mechanism *is* the turn sequence: a single-shot prompt cannot express “critique the draft you just wrote.”

4. Model Inference and Provider Architecture (`src/llm/providers.py`, `run_exp4_grid.py`)

Conversational strategies needed genuinely new provider machinery: `generate_turn` and `generate_conversation` drive a multi-message transcript, cached **turn-by-turn** on the whole running history (not just the latest message), so a resumed run replays a cached turn-1 answer byte-for-byte before even attempting a live turn-2 call. This was added *additively* — the retry/backoff/quota loop was extracted into one shared `_execute_request` both the single-turn and multi-turn call paths use, so Experiment 3's existing `_call_api/generate` kept their exact signature and behavior (its 42-test suite passed unmodified) and Experiment 3's on-disk cache stayed valid. The grid runner (`run_exp4_grid.py`) mirrors Experiment 3's two-layer resumability and quota-stop policy, with `AugmentedContext` built **once per PR** (it is tier-independent; only which slots a tier shows varies) and live-call counting per actual HTTP call, since a conversational cell costs 0, 1, or 2 calls depending on what is cached.

5. Result Scoring (`src/llm/score_exp4.py`)

Reuses Experiment 3's metric primitives directly (imported, not reimplemented) so nothing drifts between the two experiments, and adds what Experiment 4 needs: merge metrics aggregated **by_tier** and **by_strategy** (the paired, same-PR comparison this small a sample needs to be readable at all); **conversation-aware inference time** that sums *both* API calls for self-reflection/multi-turn rather than only the final turn's; a **generated-comment characteristics** proxy (comment count/length) for when no human gold comment exists; and a `compare_exp3` view on the three strategies both experiments share.

3 Experimental Results

Requirement: Describe the results obtained from the experiment. Basic content should include:

0. Scope of the Run, and the One Caveat to Read First

The full grid over all 5 tiers $\times$ 5 strategies $\times$ 2 tasks, minus the one undefined `diff+multi_turn` cell per task, is **48 cells/PR** — but up to **66 live API calls/PR** on a cold cache, since self-reflection and multi-turn each cost 2 calls. Against Groq's shared 500,000-token/day budget (already partly consumed by Experiment 3's own grid the same cal-

endar day, since both share one account), the run (`python -m src.llm.run_exp4_grid --pr-sample 8`) completed **3 PRs fully** (all 48 cells: `dotnet/aspnetcore` and `dotnet/runtime`, both merged; `home-assistant/core`, not merged) plus a **4th PR partially** (4 diff-tier merge-prediction cells), **148/384 cells, 0 failures**, before the shared daily cap stopped it cleanly (`reports/exp4_grid_run_status.md` has the full accounting).

The caveat: at $n=3-4$ PRs, per-cell accuracy is genuinely coarse (it takes only a few discrete values), so every number below is read as a **directional, paired** observation — the *same* few PRs seen across every tier and strategy — not a converged estimate. **No** complete PR carries a substantive human review comment (a real property of merged agent-authored PRs: an AI agent opens it, a maintainer merges it, often with no line comments), so BLEU/ROUGE cannot be computed here; review quality is instead shown via generated-comment characteristics and turn-by-turn qualitative examples. The `DIFF_ISSUE/COMPLETE` tiers were also run **without** `--fetch-issues` (an oversight caught only after the run completed and quota was gone), so their issue slot is empty in this data — named here plainly rather than discovered silently.

1. Examples of Different Context Constructions

Real example, `dotnet/aspnetcore` PR “[*Blazor*] *Rename EnvironmentBoundary component to EnvironmentView*” (merged):

```
-- diff (truncated) --
--- src/Components/Web/test/EnvironmentViewTest.cs (renamed)
@@ -9,7 +9,7 @@
     namespace Microsoft.AspNetCore.Components;
-public class EnvironmentBoundaryTest
+public class EnvironmentViewTest

-- diff_repo_context prepends --
Files touched by this change (repository-level view):
- src/Components/Web/test/EnvironmentViewTest.cs (renamed)
  enclosing region(s) (git hunk headings): namespace
  Microsoft.AspNetCore.Components;; public void
  ShowsContentWhenNoIncludeOrExcludeIsSpecified(string
  includeAttribut; ... [truncated for length]

-- diff_issue prepends --
Linked issue:
(no linked issue information available) [--fetch-issues not used this run]

-- complete_se_context additionally prepends --
On a past PR in dotnet/aspnetcore (“[blazor][wasm] Fix hot reload IL
trimming”), a reviewer commented: “I think the trimmer needs to find
a constant to do it's job...”
```

2. Examples of Different Prompt Designs

Same PR, Merge Prediction task, `diff_repo_context` tier, strategy `self_reflection` — the model’s actual recorded two-turn transcript:

```
TURN 1 (draft): DECISION: MERGE CONFIDENCE: 1.0

TURN 2 (self-critique): "Upon re-examining the pull request, I realize
that I may have over-weighted the change's completeness and
under-weighted its risk... My initial confidence of 1.0 was likely
anchored on the fact that most PRs do merge, rather than a careful
evaluation of the change itself... Upon further reflection, I believe
that the maintainers ... would likely have reviewed this change
carefully and potentially closed it..."

-> DECISION: CLOSE CONFIDENCE: 0.8
```

This example is deliberately **not** a flattering one: the ground truth is **MERGE**, and self-reflection’s critique turn talked itself into the *wrong* answer on this specific PR. It is included precisely because it is honest evidence of the mechanism measured in §5/§6: self-reflection systematically trades majority-class precision for minority-class recall (§6), and this transcript shows that trade happening on one concrete PR, not just in an aggregate table.

3. Merge Prediction Results

Context Tier	Acc.	Rec(NM)	Macro-F1	n
diff (baseline)	0.563	0.00	0.360	16
diff + PR description	0.600	0.20	0.489	15
diff + repository context	0.533	0.40	0.498	15
diff + issue ¹	0.467	0.40	0.444	15
complete SE context	0.600	0.00	0.375	15

Rec(NM) = not-merged recall, pooled over strategies on the same PRs. **Repository context is what moves the needle**: not-merged recall goes from **0.00** on the bare diff to **0.40** once repo context is added, and macro-F1 peaks there (0.498) — direct evidence the surrounding code lets the model see that a locally-plausible AI change does not actually fit. The **complete** tier, which stacks *everything*, regresses back to **0.00**

¹Run without `--fetch-issues`; its 0.40 recall reflects the tier’s other wiring (description, largest-churn diff ordering), not real issue text.

not-merged recall: piling all context into one prompt does not help monotonically on this sample, and may dilute the specific signal repository context alone supplied.

4. Examples of Automatically Generated Code Review Comments

Same PR, tier `diff_pr_description`, strategy `self_reflection` — turn-by-turn (full set in `results/tables/exp4_qualitative.md`):

DRAFT (turn 1):

- Please provide a more detailed diff of the changes.
- Review the test cases in `EnvironmentViewTest.cs` to ensure they still cover the correct scenarios.
- Provide a more detailed changelog or release notes for this change.

FINAL, after self-critique (turn 2):

- The use of the `@code` block in the `EnvironmentView` component may make the code harder to read and maintain. Consider using a more explicit approach to set up the conditional rendering logic.
- The use of the `@if` directive ... may lead to unexpected behavior if the child content is not properly initialized...
- The `EnvironmentView` component is using the `Microsoft.AspNetCore.Components.Web` namespace, but this may not be the correct namespace to use. Consider reviewing the documentation...

The draft is generic (“provide a more detailed diff” — of a diff already fully visible). The critique turn explicitly discards that and produces comments anchored to actual code constructs (`@code`, `@if`, the namespace). In a separate example on the same PR (a different tier), self-reflection’s critique catches a genuine typo the draft missed (`includeAttribut` → `includeAttribute`) — the same hunk heading visible in §1’s repository-context excerpt above.

5. Performance Comparison — Code Contexts

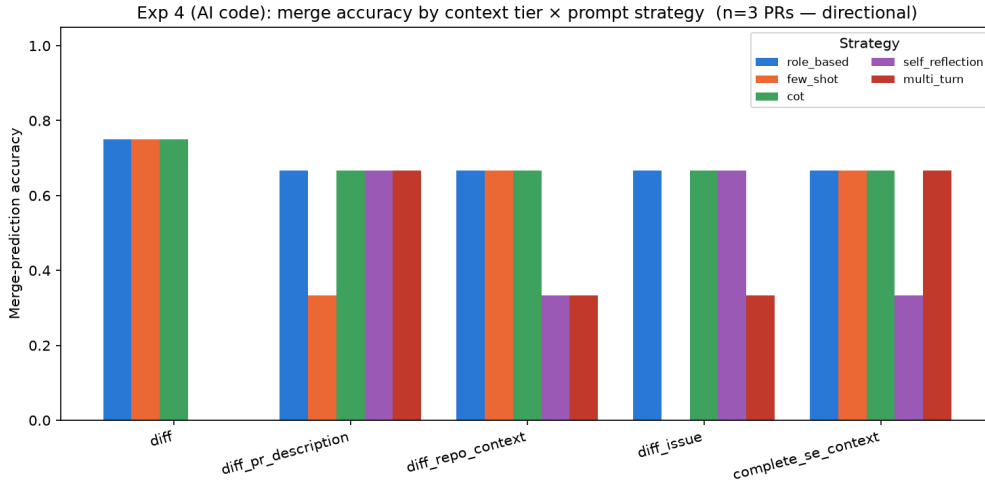


Figure 1. Merge-prediction accuracy, every tier × strategy cell ($n=3$ PRs — directional).

The context ranking is already given in §3’s table. The one addition worth stating here: context and strategy **interact** rather than acting independently — `diff_metadata`-style richness alone does not explain the pattern, since the *same* repository-context tier drives both self-reflection and multi-turn to their highest recall, while role-based/few-shot/CoT stay at 0.00 recall on *every* tier including that one (§6). The context axis and the strategy axis are each necessary but neither is sufficient alone.

6. Performance Comparison — Prompt Designs

Strategy	Acc.	Rec(NM)	Macro-F1	Calls/cell
role_based	0.688	0.00	0.407	1
cot	0.688	0.00	0.407	1
few_shot	0.500	0.00	0.333	1
self_reflection	0.375	0.60	0.375	2
multi_turn	0.500	0.50	0.486	2

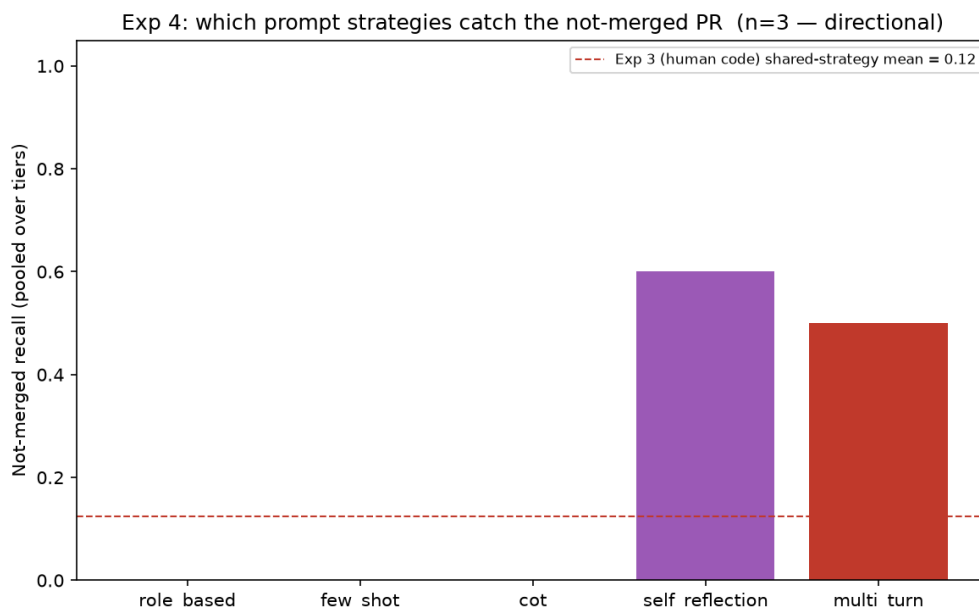


Figure 2. Which strategies ever catch the not-merged AI-authored PR, against Experiment 3’s shared-strategy mean (0.125) on human-written code.

The headline of the whole experiment: **role_based**, **few_shot**, and **cot** — the three strategies carried over from Experiment 3 — score exactly 0.00 not-merged recall on every context tier, with no exception. Only the two new strategies ever catch the not-merged AI PR at all. **self_reflection** trades the most accuracy for the highest recall (0.60): its critique turn makes it more skeptical, catching negatives at the cost of flipping some correct MERGE calls to CLOSE (§2’s transcript is exactly this). **multi_turn** is the best-balanced: highest macro-F1 (0.486) with substantial recall (0.50) and a smaller accuracy cost, because revealing context mid-conversation lets it revise a diff-only “merge” specifically when the wider context contradicts it. Inference cost roughly doubles for the two new strategies (742/677 ms mean vs. 290–602 ms for the single-shot three, since each spends 2 API calls, not 1) — a real price for the recall gain.

Review-comment **verbosity** (a proxy, since no BLEU/ROUGE gold exists on this sample): **role_based** is far the most verbose (8.5 comments/review, 1,232 chars), matching its persona-driven priority list; **cot** the tersest (3.1 comments); **multi_turn** (5.7) exceeds both single-shot alternatives it is compared against.

7. Comparative Analysis Against Experiment 3

On the three strategies both experiments ran, over their respective complete samples (Exp 3: 16 PRs, 14:2 merged; Exp 4: 3 PRs, 2:1 merged):

Strategy	Exp 3 Acc.	Exp 4 Acc.	Exp 3 Rec(NM)	Exp 4 Rec(NM)
role_based	0.891	0.688	0.125	0.00
few_shot	0.750	0.500	0.250	0.00
cot	0.875	0.688	0.000	0.00

Raw accuracy is higher on human code, but that is substantially the different base rates (87.5% vs. 67% merged) — in both experiments these three strategies barely exceed always-predict-merge, so accuracy cannot show a real capability gap. The defensible cross-experiment claim is sharper: on **human** code these same three strategies *do* occasionally reach the minority class (**few_shot** averages 0.250 recall, **role_based** 0.125). On **AI-authored** code they score **0.00 on every single tier**, and it took Experiment 4’s new strategies to catch the not-merged AI PR at all. That is the numeric confirmation of the guide’s §4.3 premise: **the prompting sufficient for reviewing human-written code stops working on AI-generated code**, which is precisely why this experiment’s context augmentation and advanced prompting exist.

4 Summary of Problems Encountered

Requirement: Primarily explain how the problems encountered during the experiment were resolved.

1. **Multi-turn conversation support did not exist yet, and adding it risked breaking Experiment 3’s already-completed, cached grid.** `CachedChatProvider.generate` was built for a single (`system`, `user`) pair; self-reflection/multi-turn need a full running transcript with an intermediate assistant reply threaded back in. *Resolution:* the retry/backoff/quota loop was extracted from `_call_api` into a shared `_execute_request`, and new `generate_turn/generate_conversation` methods were added *alongside* the untouched single-turn path, cached on the whole transcript via a separate key function (`_cache_key_from_messages`) so it can never collide with an Experiment-3 cache entry. Verified: Experiment 3’s existing 42-test provider suite passed unmodified after the change.
2. **The richest tier (`complete_se_context`) stacks five sections on the diff, and self-reflection/multi-turn’s second call resends the whole first turn — both risk exceeding Groq’s 6,000-token hard per-request cap far more easily than Experiment 3’s single-section contexts did.** *Resolution:* tighter, separately-tuned character caps (`run_exp4_grid.py`’s `GROQ_RUN_*`

constants) than Experiment 3's own, calibrated against a real offline measurement (the richest tier's turn-1 prompt came to ~4,000 characters, ~1,000 tokens, at the module defaults, comfortably under budget even doubled for a turn-2 resend); the actual run recorded 0 413 failures across 148 cells.

3. **--fetch-issues was accidentally omitted from both grid invocations, caught only after the run completed and the shared daily quota was already exhausted.** The issue-fetch feature itself was built and unit-tested, and separately validated end-to-end against the real GitHub API on a real AI PR (a genuine issue body was fetched and rendered) — but neither the pilot nor the full run actually passed the flag, so the `diff_issue/complete` tiers in the collected data carry no real issue text. *Resolution:* named plainly in `reports/exp4_grid_run_status.md` and §0/§3 above rather than silently reported as if the issue tier had been tested; a future top-up run with the flag set is the concrete remaining step.
4. **No complete PR in the sample carries a substantive human review comment, so BLEU/ROUGE — Experiment 3's headline review-quality metric — is not computable here at all.** This is a real property of the population, not a bug: merged agent-authored PRs are frequently approved with zero line comments. *Resolution:* `score_exp4.py` degrades gracefully (an explicit `note` field rather than a misleading zero), and review quality is instead assessed through a generated-comment characteristics proxy plus turn-by-turn qualitative examples (§4/§6 above), which turned out to be more informative than an n-gram score would have been on 3 PRs regardless.
5. **The lab guide's own text names “Experiment 5” as this experiment's comparison point in three separate places (§4.7.1, §4.7.5, §4.4.1), which does not exist in a four-experiment course.** Read literally it would block §4.8(7)'s required comparative-analysis deliverable entirely. *Resolution:* §4.4.1's own sentence (“optimizing model inputs rather than modifying the model itself”) unambiguously describes the Experiment-3-to-Experiment-4 relationship, so “Experiment 5” is resolved as **Experiment 3** throughout — documented in §1 above and applied consistently in `score_exp4.py`'s `compare_exp3`.
6. **The daily Groq quota is shared across the whole account, not reserved per experiment.** Running Experiment 4's grid the same calendar day as Experiment 3's own grid meant Experiment 4's run hit the 500,000-token daily cap after only 165,095 tokens of its *own* usage — roughly 334,000 tokens of that day's budget had already gone to Experiment 3. *Resolution:* this is named as a real

resource-sharing constraint in `reports/exp4_grid_run_status.md`, and the existing quota-stop/resume machinery meant no work was lost, only deferred to a later day.

5 Reflections on the Experiment

(1) Why does repository-level context improve code review performance?

An AI coding agent generates a change by pattern-matching on the immediate task, with no guarantee it has correctly modeled how that change fits the rest of the codebase — exactly the failure mode §4.3 names: “relying solely on local modifications ... is frequently insufficient.” Our numbers show this concretely: adding repository context (co-changed files, enclosing function/class names from git hunk headings) moves not-merged recall from **0.00** to **0.40** and macro-F1 to its **peak** (0.498) among all five tiers (§3). The mechanism is visible in the self-reflection transcript in §2: once the model considers the change in a slightly wider frame, it explicitly reasons about whether the maintainers would find the rename “unnecessary” — a judgment the bare diff alone does not prompt. Interestingly, the effect is *not* monotonic: stacking **every** context source (`complete_se_context`) regresses back to 0.00 recall, suggesting that more information does not automatically translate to better judgment once a model this size has to weigh five simultaneous sections — a genuinely open question for a larger sample and a larger model to resolve.

(2) Which prompt design is best suited for reviewing AI-generated code? Why?

`multi_turn`: it has the highest macro-F1 (0.486) of any strategy and the second-highest not-merged recall (0.50), without the accuracy collapse `self_reflection` suffers (0.375). The reason is mechanistic, not incidental: `multi_turn`’s first turn commits to a diff-only judgment, and its second turn reveals the broader context and asks the model to *reconsider specifically in light of it* — so it only revises when the wider context actually contradicts the local view, rather than second-guessing indiscriminately. `self_reflection` is close behind on recall (0.60, the single highest) but pays for it in accuracy, because its critique prompt asks the model to be skeptical of *its own draft* regardless of whether that draft was actually wrong — §2’s transcript shows this exact failure mode on a PR where the draft was right and the critique talked it into being wrong. Between the two, `multi_turn`’s targeted, context-triggered reconsideration is the more suitable mechanism for AI-generated code specifically, because the whole premise of §4.3 is that the local diff is *sometimes* misleading, not *always* — and `multi_turn` is the strategy built to distinguish the two cases rather than treat every draft with equal

suspicion.

(3) What specific issues are addressed by prompt optimization and context augmentation, respectively?

They fix different failure modes, and the design deliberately keeps the two axes separable (`Exp4ContextTier` vs. `Exp4Strategy`) so this is attributable rather than confounded. **Context augmentation** fixes an *information* deficit: the model cannot judge what it has never been shown, and §1/§3 demonstrate that repository context supplies exactly the missing evidence (does the change actually fit the surrounding code?) that the diff alone omits. **Prompt optimization** fixes a *disposition* deficit: even given rich context, role-based/few-shot/CoT never once predicted a not-merged AI PR correctly (§6) — the model was not *lacking information*, it was defaulting to “looks locally fine → merge” regardless of context richness. Self-reflection and multi-turn address that disposition directly, by forcing a second, explicitly skeptical pass. The clean confirmation that these are distinct levers: the *same* context tier (repository context) drives self-reflection/multi-turn to their best recall while leaving role-based/few-shot/CoT at 0.00 on that identical tier — richer information without a disposition to use it skeptically accomplishes nothing on this sample.

(4) Are different prompt combinations applicable to all code review tasks?

Partly, and unevenly. For **merge prediction**, the ranking is unambiguous: self-reflection and multi-turn are strictly necessary to catch the minority class at all (§6); role-based/few-shot/CoT are structurally incapable of it here. For **review-comment generation** the picture is different: `role_based` is the most *verbose* strategy (8.5 comments/review) by a wide margin, driven by its explicit priority-order persona, while the two new strategies sit in the middle of the pack on verbosity (§6) — their advantage in the qualitative examples (§4) is *specificity* after the critique/reveal turn, not sheer comment count. So the best strategy is genuinely **task-dependent**: a system built on this evidence would pick multi-turn (or self-reflection) for the merge-prediction path specifically to protect not-merged recall, while review-comment generation is a matter of taste between verbosity (role-based) and the sharper, context-triggered specificity multi-turn/self-reflection add on their second turn — not a case where one strategy dominates both tasks, echoing the same task-dependence Experiment 3’s own reflection (`reports/lab3.tex` Q3) found between merge prediction and review generation there.

(5) Regarding AI-generated code, what other aspects could be explored to further enhance the effectiveness of code reviews?

Directly from this run’s own honest gaps, in priority order: **(a)** actually populate the issue tier — `--fetch-issues` exists, is tested, and was validated end-to-end, but was

not enabled in this run, so the issue tier’s real effect (as opposed to its “no data” baseline) is still unmeasured; **(b)** a real repository-level fetch (full file bodies, an actual call graph) rather than the current lightweight git-heading approximation, to test whether the `diff_repo_context` tier’s already-strong result (§1/§3) gets stronger with genuine surrounding code rather than a proxy for it; **(c)** a larger, less free-tier-bounded sample — every finding here rests on 3–4 PRs, enough to see a consistent *direction* but not to fix a *magnitude*, and the `complete`-tier regression (§1/§3) in particular needs confirming rather than taking on faith; **(d)** a stronger base model than 8B, whose generated reviews are frequently generic or mildly speculative about code it cannot see (§4); and **(e)**, matching the guide’s own §4.10 suggestions, retrieval-augmented context sourced from the live repository and multi-agent or tool-using review frameworks, which could make the repository-context tier’s approximation into something closer to what a real human reviewer actually consults.

Instructor’s Comments:

Date: